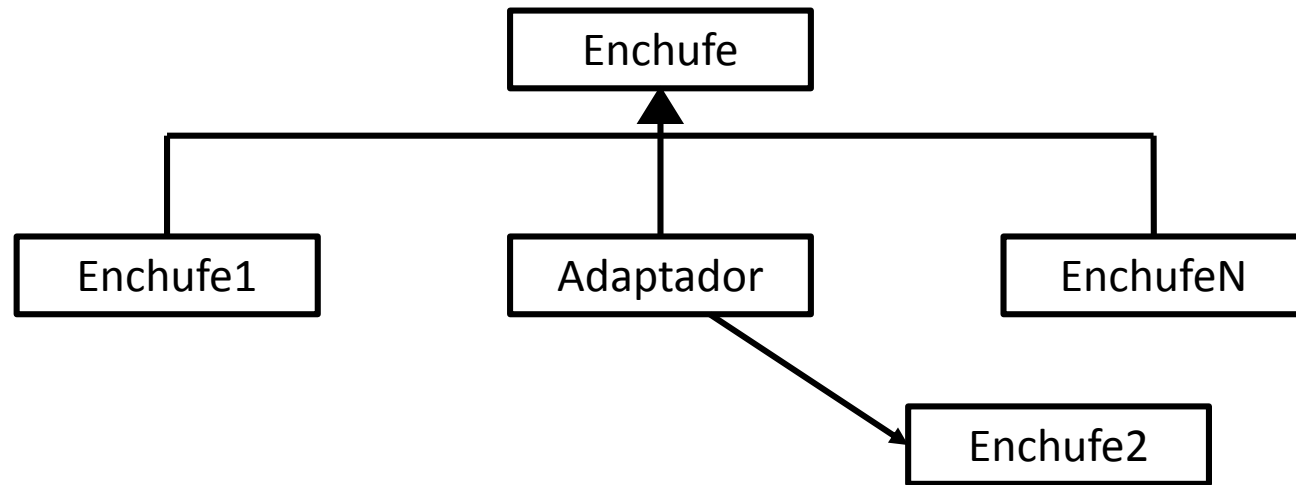


Adapter

Hace trabajar juntas a cosas incompatibles, es decir, con nombres de métodos distintos, hace de traductor, funcionando como intermediario.

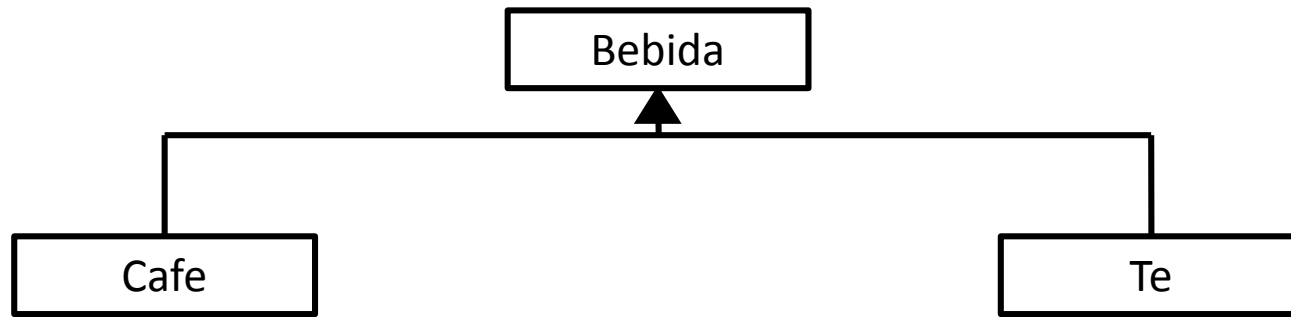


*Se puede extender el adaptador de enchufe1 y no crear “Enchufe”

- Interface Enchufe:
conectar()
- Clase Enchufe1 implements Enchufe:
conectar()
print(Enchufe1 conectado)
- Clase **Enchufe2**:
enchufar()
print(Enchufe2 enchufado)
- Clase Adaptador implements Enchufe:
private **enchufe2** e2;
public adaptador (enchufe2 e2)
this.e2=e2
conectar()
e2.enchufar()

Template method

Una clase define métodos pero deja que la subclases los modifiquen de acuerdo a las variaciones que necesiten. No queda código repetido.



- Clase Bebida:
 - Preparar()
 - Herviragua()
 - Poneringrediente()
 - Servir()
 - AgregarExtra()
 - HervirAgua()
 - Print(HirviendoAgua)
 - Servir()
 - Print(Sirviendo)
 - PonerIngrediente()
 - AgregarExtras()

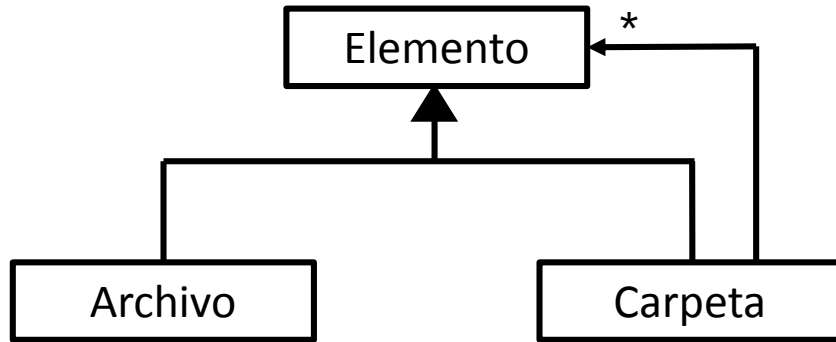
- Clase Te extends Bebida:
 - Poneringrediente()
 - Print(Poniendo saquito)
 - AgregarExtras()
 - Print(Agregando limon)

- Clase Cafe extends Bebida:
 - Poneringrediente()
 - Print(Poniendo café)
 - AgregarExtras()
 - Print(Agregando leche)

```
Bebida te=new Te()
te.preparar()
```

Composite

Permite componer objetos con otros objetos, queda algo como un árbol (Partes intermedias que pueden contener a otras partes “Finales” u hojas. Los finales y los intermedios pueden hacer lo mismo, pero los segundos pueden tener que preguntarle a los finales.

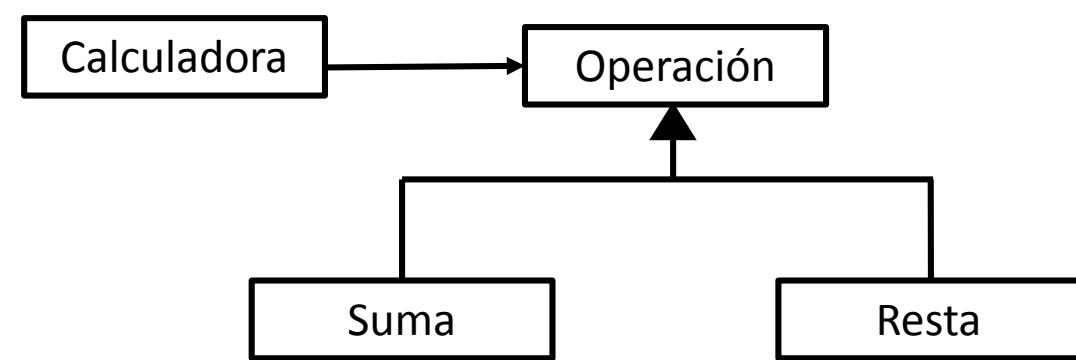


- Interface Elemento
obtenerTamaño()
mostrar()
- Clase Archivo implements Elemento:
nombre
tamaño
constructor
obtenerTamaño()
return tamaño
mostrar()
print("Archivo: "+ nombre+" Tamaño: "+ tamaño)

- Clase carpeta implements Elemento:
lista <Elemento> es= new arrayList<>()
constructor
agregar(Elemento e)
es.add(e)
obtenerTamaño()
return es.stream().maptoint
(Elemento:obtenerTamaño).sum()
mostrar()
print("Carpeta: "+ nombre);
es.stream().foreach(e.->e.mostrar())
- Clase main
Archivo a1 = new Archivo("foto.jpg", 1500)
Carpeta subcarepta = new Carpeta ("Fotos")
Subcarpeta.agregar(A1)
Carpeta base = new Carpeta ("base")
base.agregar(subcarpeta)
base.mostrar()

Strategy

Permite hacer algo de varias maneras distintas (Estrategias). Se pueden cambiar la manera que se quiere usar y agregar nuevas.



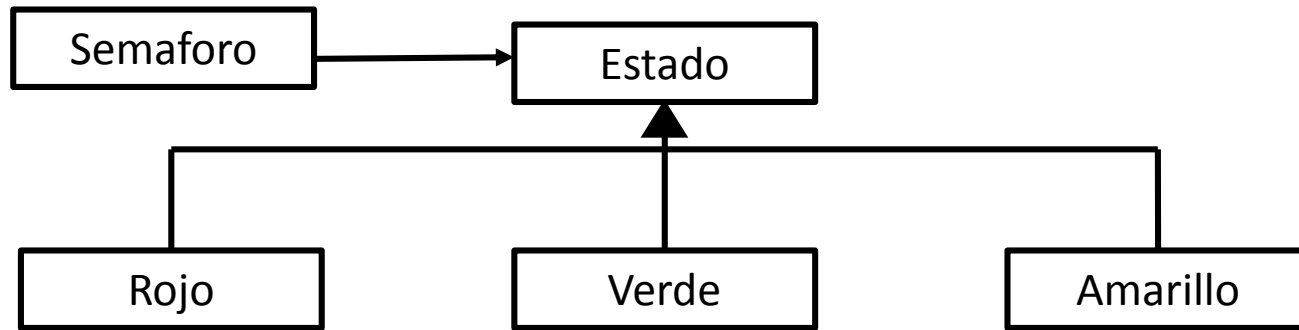
- Clase calculadora:
operación op
setOperacion (operación op)
 this.op=op
calcular(a, b)
 return op.ejecutar(a, b)

- Interface operacion:
ejecutar(a, b)
- Clase suma implements operacion:
ejecutar(a, b)
 return (a+b)
- Clase resta implements operacion:
ejecutar(a, b)
 return (a-b)

State

Permite que un objeto cambie de comportamiento cuando cambia su estado. Depende en que estado está puede cambiar a otros estados, para lo cual se sustituye el objeto de estado activo por otro con el nuevo estado.

Diferencia con strategy: acá los estados pueden conocerse y cambiar entre si, las estrategias no.



- Clase semáforo

```
EstadoSemaforo Estado
Semaforo()
    this.estado=New estadoRojo()
setEstado(EstadoSemaforo estado)
    this.estado=estado
Mostrar()
    estado.mostrar()
Cambiar()
    estado.cambiar(this)
```

- main

```
Semaforo semáforo=new semáforo()
For 1:= to 6
    semáforo.mostrar()
    semáforo.cambiar()
```

- Interface EstadoSemaforo:

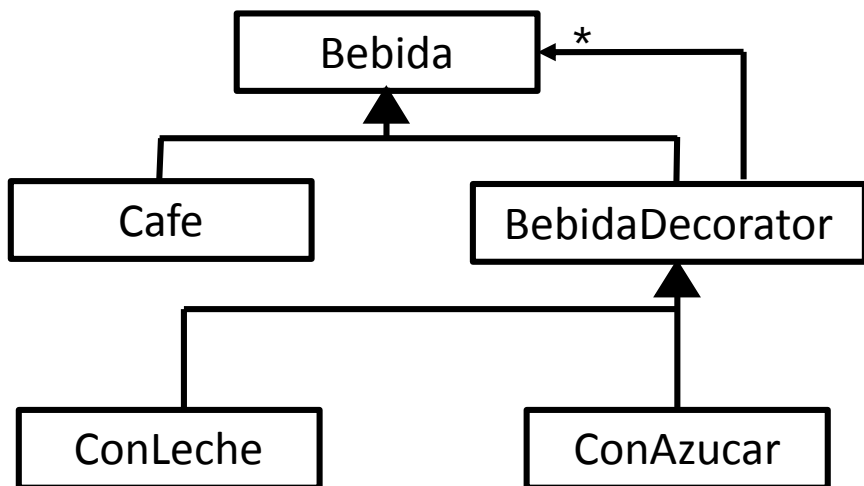
```
cambiar(Semaforo S)
mostrar()
```

- Clase EstadoRojo implements EstadoSemaforo

```
cambiar(Semaforo S)
    semáforo.setEstado(new EstadoVerde)
    //El verde pasa a amarillo y el amarillo al rojo
mostrar()
    print (Rojo, detenerse)
```

Decorator

Mete un objeto dentro de otro para añadirle funcionalidades.



- Interface **Bebida**:
getDescripcion()
getCosto()

- Clase Cafe implements **Bebida**:
getDescripcion()
return "Café"
getCosto()
return 10,0

- Clase BebidaDecorator implements **Bebida**:
Bebida bebida
constructor

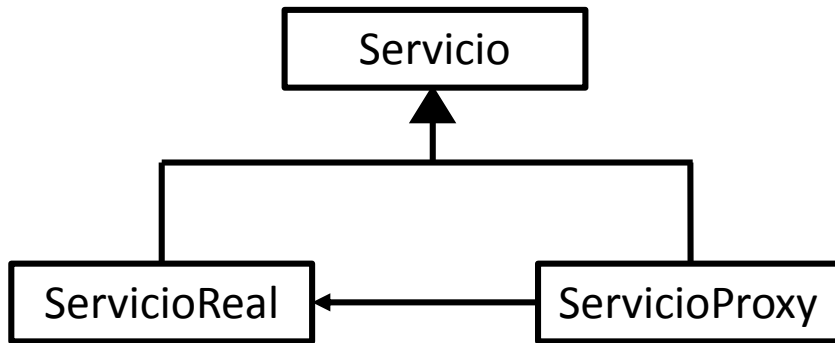
- Clase ConLeche extends BebidaDecorator:
ConLeche(**Bebida** bebida)
super(bebida)
getDescripcion()
return bebida.getDescripcion()+"con leche"
getCosto()
return bebida.getCosto()+2,0

- Clase Cagetería
Bebida cafesimple= new Café()
print(cafesimple.getdescripcion()+" sale "+cafesimple.getcosto())
Bebida cafeconleche= new conleche(cafesimple)
print(cafeconleche.getdescripcion()+" sale
"+cafeconleche.getcosto())

Si se hace un "con azúcar" se le pasa el "Con leche"

Proxy

Sustituye al objeto original y controla el acceso a este. Sirve si hay que regular el uso de un objeto que consume muchos recursos. Se puede ejecutar algo antes o después de que se ejecute una acción en el objeto original. Al recibir solicitud de un cliente el proxy crea el objeto real y le delega el trabajo.

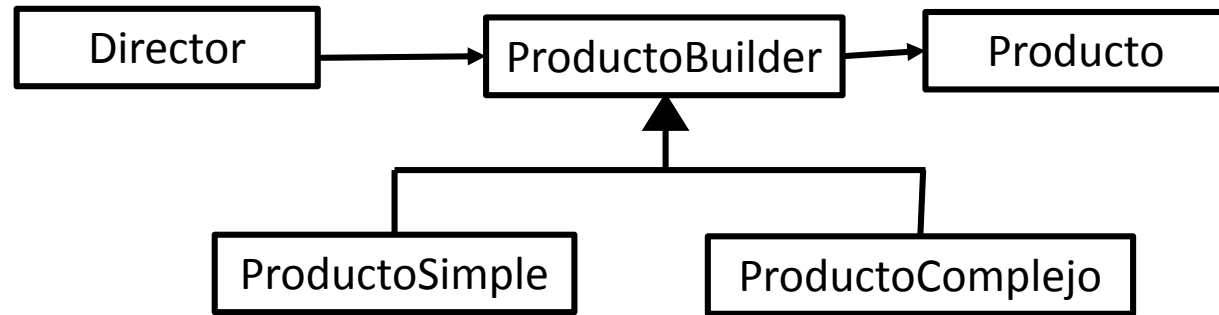


- Interface Servicio:
usar()

- Clase Servicio Real implement Servicio:
usar()
 print(Sistema en uso)
- Clase ServicioProxy implements Servicio:
 ServicioReal servicioReal
 Boolean autenticado
 constructor
 autenticar(string Clave)
 if ("secreta".equals(Clave):
 autenticado=true
 usar()
 if autenticado
 servicioReal.usar()
 else
 print(Acceso denegado)

Builder

Construir objetos paso a paso. Distintos tipos y representaciones. Inicialización de muchos campos. Sin un constructor enorme. No necesitan invocar todos los pasos, solo los necesarios. Varias clases construyen pero distinto. La clase directora no siempre es necesaria.

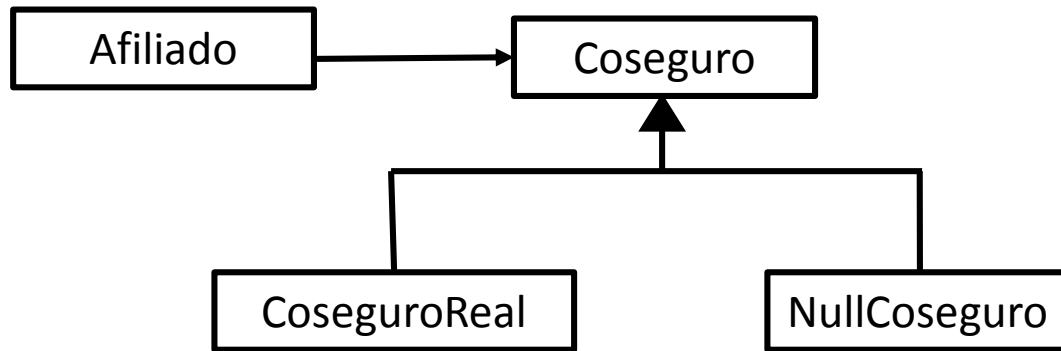


- Clase productoSimple implementa ProductoBuilder:
construirParteA()
 producto.SetParteA(ParteASimple)
construirParteB()
 producto.SetParteB(ParteBSimple)
- Clase productoComplejo implementa ProductoBuilder:
construirParteA()
 producto.SetParteA(ParteAComplejo)
construirParteB()
 producto.SetParteB(ParteBComplejo)
- Clase Producto:
 parte1, parte2: String
 setParteA(PA)
 this.parteB=PA
 setParteA(PB)
 this.parteB=PB
- Clase ProductoBuilder:
 Producto producto
 constructor
 construirParteA()
 construirParteB()
 ObtenerProducto()
 return Producto
- Clase director:
 constuirProducto(ProductoBuilder builder)
 buiden.ConstruirParteA()
 buiden.ConstruirParteB()

Null object

Reemplaza las secciones del código donde se pregunta por null con un objeto que muestra el comportamiento por defecto.

A partir de la clase en cuestión se crea una subclase de objeto nulo. Ambas tienen el método `isNull()` que devuelve `true` para el nulo y `false` para el real, u otro comportamiento adecuado.



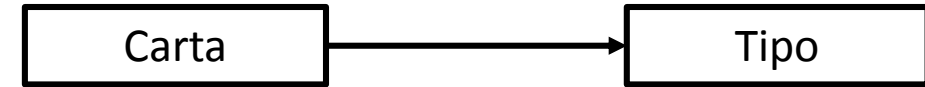
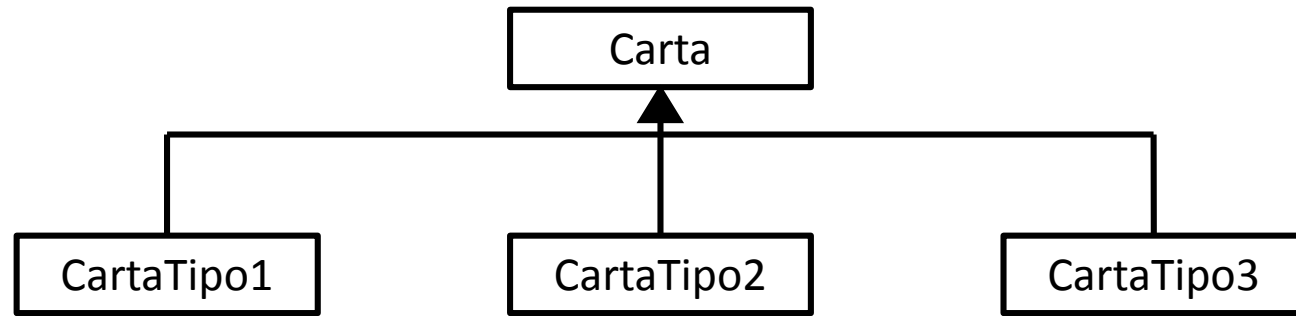
- Interface `Coseguro`:
`calcularDescuento(monto)`
- Clase `CoseguroReal` implements `Coseguro`:
`private double porcentaje`
`calcularDescuento(monto)`
`return monto*porcentaje;`
- Clase `NullCoseguro` implements `Coseguro`:
`calcularDescuento(monto)`
`return 0.0;`

- main:

```
Afiliado afiliadoConCoseguro = new Afiliado (new Afiliado (New CoseguroReal ()))
Afiliado afiliadoSinCoseguro = new Afiliado (new Afiliado (New NullCoseguro()))
```

Type object

Los objetos tienen tipos o categorías dinámicas. Los tipos son instancias de tipo. En vez de tener una herencia gigante (Si hubieran 50 tipos de algo). Solo hay un atributo tipo en el objeto.



Y puede cambiar en tiempo de ejecución